



TRABALHO PRÁTICO 2: ARIANAGRAM

Redes de Computadores Junho/2026

Dados do Trabalho

Título do trabalho: Trabalho Prático 2: Arianagram

Alunos: Thales Gregory Vasconcelos Santos – 2018019630

1 Introdução

Este trabalho tem como objetivo apresentar o desenvolvimento do Arianagram, uma rede social minimalista baseada no modelo cliente-servidor, utilizando comunicação via sockets TCP e suporte a múltiplas conexões simultâneas. A proposta central do sistema é permitir que diferentes usuários se conectem a um servidor, publiquem mensagens, sigam outros usuários e recebam notificações em tempo real quando os perfis acompanhados realizarem novas postagens.

Ao longo do desenvolvimento, foram aplicados conceitos importantes de redes de computadores, como criação e configuração de sockets TCP, uso de IPv4 e IPv6, envio e recebimento de mensagens binárias de tamanho fixo, tratamento de conexões encerradas, sincronização de estruturas compartilhadas e comunicação assíncrona entre servidor e clientes.

2 Arquitetura do Sistema

2.1 Visão Geral

Nesse sistema, o servidor concentra a lógica principal da aplicação, enquanto os clientes funcionam como interface de interação, enviando comandos e exibindo mensagens recebidas.

A arquitetura do servidor segue o modelo Single-Process, Multi-Thread. O processo principal permanece aguardando novas conexões e, a cada cliente conectado, cria uma thread dedicada para atender aquela sessão. Isso permite que múltiplos usuários interajam simultaneamente com o sistema.

Além do fluxo tradicional de requisição e resposta, o Arianagram também implementa notificações push, explorando o paradigma Publish-Subscribe. Assim, quando um usuário publica uma mensagem, o servidor verifica seus seguidores ativos e envia a postagem diretamente para os clientes conectados que seguem aquele perfil.

2.2 Estruturas de Dados

No servidor, foram utilizadas estruturas de dados simples para armazenar as informações necessárias ao funcionamento do sistema. As principais estruturas estão relacionadas ao feed global de mensagens e ao controle de seguidores.

A estrutura do feed, implementada em `feed.c`, funciona como um buffer circular. Ela armazena as mensagens publicadas pelos usuários, mantendo um contador de mensagens válidas e uma posição inicial para indicar a mensagem mais antiga. Quando o limite de armazenamento é atingido, as mensagens mais antigas passam a ser sobrescritas pelas novas publicações. Essa abordagem permite manter o histórico recente de mensagens de forma simples e eficiente.

Já a estrutura de seguidores, implementada em `followers.c`, mantém a relação entre usuários seguidos e seus respectivos seguidores. Para cada usuário, é armazenada uma lista de perfis que o

seguem. Assim, quando um usuário realiza uma postagem, o servidor consulta essa estrutura para identificar quais clientes devem receber uma notificação MSG_PUSH.

Além disso, os nomes de usuário são normalizados antes do armazenamento, removendo o caractere @. Essa decisão facilita as comparações entre usuários e também permite tratar corretamente casos como seguidores duplicados e auto-follow, que são ignorados silenciosamente pelo sistema.

2.3 Funcionamento da Thread no Servidor

A arquitetura do servidor segue o modelo Single-Process, Multi-Thread. O processo principal permanece aguardando conexões e, sempre que um novo cliente é aceito com `accept()`, cria uma thread dedicada para atender aquela conexão.



Figura 1: Ciclo de vida de uma thread no servidor

O uso de `pthread_detach()` permite que os recursos da thread sejam liberados automaticamente ao final da execução, sem a necessidade de utilizar `pthread_join()` no processo principal.

Dentro da função `client_thread`, a thread recebe inicialmente a mensagem MSG_CONNECT, enviada pelo cliente logo após a conexão. A partir dessa mensagem, o servidor identifica o usuário conectado e registra esse cliente na lista de clientes ativos.

```

int status = recv_all(client_fd, &message, sizeof(message));

if (status != 0) {
    close(client_fd);
    return NULL;
}

memcpy(username, message.username, USER_SIZE);
username[USER_SIZE] = '\0';
printf("[CONN] %s%s conectou.\n", (username[0] == '@') ? "" : "@", username);

active_clients_add(client_fd, username);

```

Figura 2: Lista de clientes ativos no servidor

Depois do handshake, a thread entra em um loop de processamento, permanecendo ativa enquanto a conexão com o cliente estiver aberta. Nesse loop, cada mensagem recebida é enviada para `handle_message()`, que trata comandos como `MSG_POST`, `MSG_FOLLOW` e `MSG_READ`.

```

for (;;) {
    status = recv_all(client_fd, &message, sizeof(message));
    if (status == 1) {
        break;
    }
    if (status < 0) {
        perror("recv");
        break;
    }

    if (handle_message(client_fd, username, &message) < 0) {
        perror("send");
        break;
    }
}

```

Figura 3: Processo de leitura de mensagens

No caso de uma mensagem `MSG_POST`, a thread armazena a publicação no feed global e aciona o envio de notificações push para os seguidores ativos daquele usuário.

```
static int handle_message(int client_fd, const char *username, const Message *message) {
    if (message->type == MSG_POST) {
        pthread_mutex_lock(&feed_mutex);
        FeedEntry entry = store_post(username, message);
        pthread_mutex_unlock(&feed_mutex);
        send_push_to_followers(&entry);
        return 0;
    }

    if (message->type == MSG_FOLLOW) {
        pthread_mutex_lock(&followers_mutex);
        followers_add(&followers, message->content, username);
        pthread_mutex_unlock(&followers_mutex);
        return 0;
    }

    if (message->type == MSG_READ) {
        return send_feed_entries(client_fd);
    }

    return send_all(client_fd, message, sizeof(*message));
}
```

Figura 4: Processo de leitura de armazenamento e push de postagens

Quando o cliente encerra a conexão, `recv_all()` detecta o fechamento do socket, a thread sai do loop, remove o cliente da lista de ativos, fecha o socket e finaliza sua execução.

```
active_clients_remove(client_fd);
close(client_fd);
printf("[DISC] %s%s desconectou.\n", (username[0] == '@') ? "" : "@", username);
return NULL;
```

Figura 5: Fechamento da thread

Assim, o ciclo de vida da thread pode ser resumido em: criação após `accept()`, identificação do cliente via `MSG_CONNECT`, registro como cliente ativo, processamento dos comandos recebidos e limpeza dos recursos ao final da conexão.

3 Desafios e Dificuldades

O principal desafio desse sistema ocorreu principalmente no envio de pushes para permitir que uma thread enviasse mensagens para sockets associados a outros clientes. Quando um usuário realiza um `MSG_POST`, a thread responsável por essa conexão precisa identificar quais seguidores estão ativos e encaminhar uma mensagem `MSG_PUSH` diretamente para os sockets correspondentes.

Para isso, foi necessário manter uma lista de clientes ativos, relacionando cada usuário conectado ao seu socket. Além disso, como essa lista, o feed global e a tabela de seguidores são compar-

tilhados entre múltiplas threads, foi necessário proteger o acesso a essas estruturas com mutexes, evitando inconsistências causadas por acessos concorrentes.

Outro ponto importante foi tratar corretamente desconexões. Quando um cliente encerra a conexão, ele deve ser removido da lista de ativos para impedir que o servidor tente enviar pushes para sockets inválidos.

Por fim, no lado do cliente, foi utilizado `select()` para monitorar simultaneamente o teclado e o socket, permitindo que notificações `MSG_PUSH` fossem recebidas de forma assíncrona, mesmo quando o usuário não estivesse digitando comandos.

4 Conclusão

Assim, concluímos que este projeto permitiu a aplicação prática de conceitos de redes de computadores, programação em C e desenvolvimento de sistemas baseados no modelo cliente-servidor. A implementação do sistema *Arianagram* proporcionou um estudo mais aprofundado sobre comunicação via sockets TCP, múltiplas conexões simultâneas, uso de threads e sincronização de estruturas compartilhadas.

Além disso, o projeto possibilitou compreender melhor o funcionamento de um sistema baseado em Publish-Subscribe, no qual o servidor não apenas responde às requisições dos clientes, mas também envia notificações assíncronas por meio de mensagens `MSG_PUSH`. Dessa forma, o desenvolvimento contribuiu para consolidar conhecimentos sobre concorrência, gerenciamento de conexões e comunicação em tempo real entre processos distribuídos.

Referências

- [1] DCC/ICEx/UFMG. Especificação do trabalho prático 2 — arianagram. Documento interno, 2026. Material da disciplina.
- [2] GeeksforGeeks. Multithreading in c. <https://www.geeksforgeeks.org/c/multithreading-in-c/>, 2025. Acesso em: 01 jun. 2026.
- [3] Marcos Augusto Menezes Vieira. Redes de computadores. Slides da disciplina DCC218 - INTRODUÇÃO A SISTEMAS COMPUTACIONAIS, Departamento de Ciencia da Computacao, ICEx/UFMG, 2026. Material de aula.